DÉPARTEMENT
de PHYSIQUE

TP N° 1 : Introduction au langage R | Prise en main

Durée esitmée | 2 **séances de** 1*h*30

I | But de la manipulation

R est un langage de programmation interactif **interprété** et **orienté objet** didié essentiellement à l'analyse des données et au statistique. En effet, il contient une très large collection de méthodes statistiques ainsi que des facilités graphiques importantes. L'apprentissage d'un nouveau langage de programmation est basée sur l'exposition à un maximum de codes et surtout sur la pratique. Ainsi, le but de cette manipulation est de se familiariser avec la syntaxe du langage **R** en exécutant et en comprenant les exemples donnés dans chaque paragraphe. Les thèmes de cette manipulation sont les suivants :

1. les objets sous **R** (`integer`, `logical`, `numeric`, `complex`, `character`,$\cdots$),
2. les classes (`voctor`, `matrix`, `array`, `list`, `data.frame`, `time.series`,$\cdots$),
3. les structures de contrôles (`if..`, `if..else..`, `if..else if ..`, *etc*),
4. les structures répétitives - boucles - (`for() {...}`, `while() {...}` ou `repeat() {...}`).

Des exercices d'application son ensuite donnés à la fin de chaque paragraphe.

II | Installation de R

R est un logiciel et un langage gratuit (*open source*) multi-plate-formes, il fonctionne sous Linux, Windows et Macintosh. Pour exécuter des codes en langage **R**, il faut installer deux composantes essentielles :

1. l'**interpréteur** de langage **R** qui est fourni par CRAN et la Fondation R (parmi d'autres). L'interpréteur est directement téléchargeable sur le site : https ://cran.r-project.org/ (la version base),
2. un IDE - Integrated Development Environment. Malgré qu'un **IDE** (GUI) est installé avec l'interpréteur **R** - version sous Windows, il est préférable d'installer un IDE tel que **RStudio** qui fourni plus de fonctionnalités. **RStudio** est téléchargeable à partir du site : https ://posit.co/download/rstudio-desktop/. Cependant, il existe une autre alternative qui consiste, après l'installation de l'interpréteur de travailler avec VSCode en ajoutant quelques extensions.

NB : l'installation se fait dans l'ordre : installer l'interpréteur puis l'IDE.

(C) Dr. ABBASSI A
Department of Physics - ISI Mahdia | UM

III | Commandes R essentielles

Voici un guide rapide des commandes **R** essentielles pour obtenir de l'aide, installer et gérer des packages :

Commande R	Description
`help("function")?`	Affiche l'aide sur une fonction spécifique.
`??function?`	Raccourci équivalent à `help("function")?`.
`help.start()?`	Lance laide HTML dans le navigateur.
`example("function")?`	Montre des exemples dutilisation dune fonction.
`args(function)?`	Affiche les arguments dune fonction.
`help.search("mot")?`	Recherche les fonctions liées à un mot-clé.
`apropos("mot")?`	Liste les objets contenant le mot-clé.

Tableau 1 – Commandes essentielles **R** : HELP

Commande R	Description
`install.packages("nom")?`	Installe un package depuis `CRAN?`.
`install.packages("devtools")?`	Installe le package `devtools?`.
`devtools::install?_github("user/repo")?`	Install un package depuis `GitHub?`.
`BiocManager::install("nom")?`	Installe un package depuis Bioconductor.

Tableau 2 – Commandes essentielles **R** : Installation des PACKAGES

DÉPARTEMENT *de* PHYSIQUE

Commande R	Description
`library(nom)?`	Charge un package installé.
`require(nom)?`	Charge un package si disponible, retourne `TRUE`/`FALSE` selon le succès.
`installed.packages()?`	Liste tous les packages installés.
`update.packages()?`	Met à jour tous les packages installés.
`remove.packages("nom")?`	Supprime un package installé.

Tableau 3 – Commandes essentielles **R** : Gestion des PACKAGES

Commande R	Description
`rm(list = ls())?`	Supprime tous les objets de l'environnement.
`ls()?`	Liste tous les objets actuellement en mémoire.

Tableau 4 – Commandes essentielles **R** : Gestion de l'ENVIRMENT

NB : Le démarrage d'une session **R** provoque automatiquement le chargement de certains packages. Par défaut, les packages suivants sont chargés automatiquement : `base`, `methods`, `datasets`, `utils`, `grDevices`, `graphics`, `stats`. Pour charger dautres packages, il faut utiliser la fonction library, comme dans cet exemple : `library(dplyr)`.

IV | Méthode de travail

Vous allez exécuter impérativement tous les différénts codes disponibles. L'objectif est de comprendre tous les résultats obtenus. Vous pouvez faire un `copier-coller` des commandes dans une fenêtre script (ou dans le console) puis les exécuter ensuite (par exemple en cliquant sur le bouton `RUN` en haut à droite de la fenêtre script ou avec le raccourci clavier `CTRL +ENTER` (sous Windows). Des exercices sont proposés à la fin de chaque partie pour vérifier que les principales notions ont été bien assimilées.

V | Les objets en R

Les éléments de base du langage **R** sont des objets qui peuvent être des vecteurs, des matrices, des tableaux de données, des séries chronologiques, des fonctions, des graphiques, etc. Les objets **R** se différencient par leur **mode**, qui décrit leur contenu, et leur **classe**, qui décrit leur structure.

1 Les différents **modes** sont :

Entier : `integer`,

Logique (ou booléen) : `TRUE | FALSE : logical`,

Numérique : `numeric`,

Complex : `complex`,

Caractère : `character`.

2 Les principales classes sont :

1 Les vecteurs : `vector`,

2 Les matrices : `matrix`[1],

3 Les tableaux : `array`,

4 Les lists : `list`,

5 Les tableaux de données : `data.frame`,

6 Les série temporelles : `time.series`.

Ainsi, on peut avoir des vecteurs, matrices, tableaux, séries temporelles de mode `integer`, `logical`, `numeric`, `complex`, `character`, mais de classes homogènes (par exemple, un vecteur ne peut pas contenir à la fois des nombres et des caractères), alors qu'une liste `list` ou un tableau de données `data.frame` les modes peuvent être hétérogènes (par exemple, on peut avoir une colonne des nombres donnant l'age des individus et une colonne donnant le sexe des mêmes indiidus).

V | 1 ▷ Opérations arithmétiques et d'affectation dans R

Saisir le script suivant dans l'environnement de RStudio :

```
1
2  # Ceci est un commentaire en langage R
3
4  3 + 4            # addition
5  5 - 6            # soustraction
6  3 * 7            # multiplication
7  (7 + 3) / 2      # division
8  2^5              # exposant
9  28 %% 6          # modulo
10 my_var <- 52     # affectation : permet de stocker une valeur dans une
11                  # variable, pour une utilisation ulterieure.
12 my_var           # permet d'afficher la valeur de la variable sur le
13                  # console.
14 my_apples <- 5
15 my_oranges <- 6
16
17 my_apples + my_oranges  # permet d'effectuer des operations
18 my_fruits <- my_apples + my_oranges # sur les variables et d'affecter
```

1. une matrice est un tableau (plus que $2D$) à $2D-$dimensions.

© Dr. ABBASSI A. Department of Physics - FS Mahdia (TN)

```
                                    # le resultat a une troisieme
                                    # variable : ici my_fruits.
```

Listing 1 – Opérateurs arithmétiques et d'affectation dans R

NB :

1. **R** n'affiche pas la valeur d'une variable dans la console après affectation. En effet, l'affectation `x <- -5` ne génère aucune sortie, car **R** suppose que aurez besoin de cette variable ultérieurement.
2. Dans l'opération `my_apples + my_oranges` vous avez pu effectuer cette opération car les deux variables contiennent des valeurs numériques. Mais que ce passe-t-il si l'une des deux variables `my_oranges <- "oranges"` ? Ceci nous amène à bien faire attention aux types des variables sur lesquelles on travaille.
3. lors de l'évaluation d'une expression mathématique il faut faire attention à l'ordre de priorité des différents opérateurs exitant dans cette dernière. En effet, le **tableau ci-dessus** résume cette remarque.

moins prioritaire	plus prioritaire	très prioritaire
`+, -?`	`*, //, %, /?`	`**, ()?`

Tableau 5 – ordre de priorité des opérateurs arithmétiques.

Saisir le script suivant dans l'environnement de RStudio :

```

# Ceci est un commentaire en langage R

my_numeric <- 4.5            # type numerique
my_character <- "universe"   # type caractere
my_logical <- FALSE          # type logique

class(my_numeric)            # donne une classe : "numeric"
class(my_character)          # renvoi une classe : "character"
class(my_logical)            # renvoi une classe : "logical"
```

Listing 2 – Types de données dans R

V | 2 ▷ *Les vecteurs*

Il s'agit de l'objet de base dans **R**. Un vecteur est une entité unique formée d'une collection ordonnée d'éléments de même mode (type homogène) (c'est-à-dire : `numeric`, `integer`, etc). La constitution manuelle d'un vecteur est réaliser grâce à la fonction `c()` : `c(e11, e12, ...)`.

Saisir le script suivant dans l'environnement de RStudio :

```

# Ceci est un commentaire en langage R

c(1, 10, 49)                    # creation manuelle d'un vecteur
numeric_vector <- c(1, 10, 49) # dans l'objet numeric_vector mettre
                                # le vecteur c(1, 10, 49)
numeric_vector                  # affiche le vecteur dans la console.
numeric_vector1 = c(4.3, -5, 18)# = peut jouer le role <-
concat_vector <- c(numeric_vector, numeric_vector1) # la fonction c
                                 # permet aussi de concatener deux
                                    vecteurs
concat_vector                    # voir le resultat.
character_vector <- c("a", "b", "c") # vecteur de caracteres
text_vector <- c("grand", "petit")
boolean_vector <- c(TRUE, FALSE, TRUE) # vecteur booleen.
```

Listing 3 – Création de vecteurs dans R

NB :

1. pour afficher le contenu d'une variable il suffit d'écrire son nom dans la console,
2. dans `RSudio` toute nouvelle variable fraîchement créée sera afficher dans la fenêtre `ENVIRONNEMENT`,
3. pour supprimer une variable il suffit de tapper la commande `rm(nom_variable)` dans la console ou utiliser le menu de la fenêtre `ENVIRONNEMENT`,
4. dans **R** les nombre décimaux doivent être encodés avec un point et non pas une virgule, les chaînes de caractères entourées par des guillemets doubles `" "`, les valeurs logiques par `TRUE | FALSE` ou `T | F` et enfin les données manquants par la chaîne de caractères `NA`,
5. malgré que les deux sympboles `<-` et `=` peuvent faire de l'affectation, mais il est fortement conseiller d'utiliser `<-` pour l'affection et laisser le symbole `=` pour l'initialisation des arguments d'une fonction.
6. Lorsque vous quittez **R**, vous avez la possibilité de sauvegarder votre espace de travail pour retrouver toutes les variables créées lors de la prochaine session. Faites le test : sauvegarder votre script qui apparaît dans l'éditeur de texte dans un répertoire approprié puis quitter **R**. **RStudio** vous demande si vous voulez sauver votre environnement de travail : répondez **oui**. Ouvrez à nouveau **RStudio** : vous retrouvez le même environnement de travail, en particulier les objets créés lors de la session **R** présente sont à nouveau disponibles.

V | 3 ▷ *Nommer un vecteur*

Avant de faire une première analyse, vous décidez tout dabord de collecter tous les gains et les pertes de la semaine dernière dans deux jeux différents :

ABBASSI A. Mahdia
of Physics - ISI

Saisir le script suivant dans l'environnement de RStudio :

```
1
2  # Ceci est un commentaire en langage R
3
4  game1_vector <- c(140, -50, 20, -120, 240) # gain et perte pendant une
5                                             # semaine pour le jeux 1
6  game1_vector                               # affichage
7  game2_vector <- c(-24, -50, 100, -350, 10) # gain et perte pendant une
8                                             # semaine pour le jeux 2
9  game2_vector                               # affichage
10
11 # Nommer un vecteur version 1 : fonction : names
12
13 # nommer le vecteur : game1_vector
14 names(game1_vector) <- c("Monday", "Tuesday", "Wednesday", "Thursday",
       "Friday")
15 game1_vector                  # affichage apres modification
16 # nommer le vecteur : game2_vector
17 names(game2_vector) <- c("Monday", "Tuesday", "Wednesday", "Thursday",
       "Friday")
18 game1_vector                  # affichage apres modification
19
20 # Nommer un vecteur version 2 : fonction : names
21
22 days_vector <- c("Monday","Tuesday","Wednesday","Thursday","Friday")
23
24 # nommer le vecteur : game1_vector
25 names(game1_vector) <- days_vector
26 game1_vector                  # affichage apres modification
27 # nommer le vecteur : game2_vector
28 names(game2_vector) <- days_vector
29 game1_vector                  # affichage apres modification
```

Listing 4 – Nommer des vecteurs dans R

Pour calculer le gain ou la perte il faut effectuer des opérations sur les vecteurs.

V | 4 ▷ *Opérations sur les vecteurs*

Saisir le script suivant dans l'environnement de RStudio :

```
1
2  # Ceci est un commentaire en langage R
3
4  numeric_vector <- c(5.25, 6.75, 1, 4.3, -5)    # creation d'un vecteur
5  numeric_vector[1]  # permet d'acceder aux composantes d'un vecteur
6  numeric_vector[0]  # Attention : l'indice du premier element est 1
7
8  # modifie la valeur d'une composante d'un vecteur
9  numeric_vector[1] <- 17.5
10 numeric_vector     # afficher pour verifier
11 index <- c(2, 4)   # que contient le vecteur index ?
12
13 # extrait les coordonnes associees au vecteur index
14 numeric_vector[index]
15 numeric_vector[c(1, 3, 5)]# appel plus compact
16
17 # le resultat de l'extraction est stocke dans extract_vector
18 extract_vector <- numeric_vector[c(1, 3, 5)]
19 x_vector <- 3:14 # la commande est utile pour creer certains vecteurs
20
21 # modification de plusieurs comosantes d'un vecteur
22 x_vector[1:5] <- 1.5
23
24 A_vector <- c(1, 2, 3)
25 B_vector <- c(4, 5, 6)
26
27 total_vector <- A_vector + B_vector # somme de deux vecteurs
28 total_vector
29
30 # Operations sur les vecteurs :
31
32 numeric_vector[1] <- 0
33 2 * numeric_vector # multiplication par 2 les composantes du vecteur
       et affiche le resultat
34 inv_vector <- 3 / numeric_vector  # remarquer que 1/0 = Inf
35 sub_vector <-  numeric_vector - 4 # enleve 4 de chacune des
       composantes du vecteur
36 numeric_vector <- numeric_vector + inv_vector
37 numeric_vector <- numeric_vector - inv_vector # remarquer que Inf -
       Inf = NaN (Not a Number)
38 numeric_vector * inv_vector  # multiplication terme a terme des deux
       vecteurs
39 inv_vector / numeric_vector  # division terme a terme des deux
       vecteurs
40 sum(numeric_vector)          # calcul la somme des composantes du
       vecteur
41 length(numeric_vector)       # donne la dimension du vecteur
42 t(numeric_vector)            # transposition du vecteur
43 g_vector <- c(sqrt(2), log(10))
44 (1 + exp(2)) / cos(8)        # R comme calculatrice
```

Listing 5 – Opérations sur les vecteurs dans R

V | 5 ▷ *Opérations logiques*

En plus des opérateurs arithmétiques classiques on dispose des opérateurs logiques dont le résultat est de la classe `logical` c'est-à-dire : `TRUE` ou `FALSE`.

```r

# Ceci est un commentaire en langage R

1 == 2   # == : realise un test d'egalite et renvoie un booleen TRUE | FALSE
1 == 1   # renvoie TRUE
1 > 2    # renvoie FALSE
1 != 2   # != signifie not equal
(1 == 1) & (1 > 2) # & signifie AND logique. Renvoie : FALSE
(1 == 1) | (1 > 2) # | signifie OR Inclusif logique. Renvoie : TRUE

numeric_vector <- c(5.25, 6.75, 1, 4.3, -5)
extract_vector <- numeric_vector[c(1, 3, 5)]
extract_vector == 5
extract_vector == -5  # test sur les composantes d'un vecteur
numeric_vector > 1 & numeric_vector <= 5

# le resultat du test est stocke dans index
index <- (numeric_vector > 1 & numeric_vector <= 5)
# utilisation de la fonction is.vector() renvoyant l'expression logique TRUE si son argument est un vecteur et FALSE sinon
is.vector(numeric_vector)
is.numeric(numeric_vector)   # renvoie TRUE
is.character(numeric_vector) # renvoie FALSE

mode(numeric_vector)  # renvoie le mode ou type : "numeric"
numeric_vector * numeric_vector  # on peut manipuler le type numeric
numeric_vector[1] <- "test"      # modifie l'element 1 du vecteur
mode(numeric_vector)             # le type change en "character"

# il affiche cette erreur : Erreur dans numeric_vector * numeric_vector :
argument non numerique pour un operateur binaire. On ne peut pas manipuler
deux vecteurs de modes character.

numeric_vector * numeric_vector
```

Listing 6 – Opérations logiques dans R

Extraction d'individus. Lorsqu'on manipule des jeux de données (série statistique) en statistique, on est souvent amené à extraire des **individus** ayant certaines caractéristiques (**modalités**) (par exemple tous les cadres de **sexe masculin** avec un âge compris entre 30 et 35 ans dans une base de données actuarielle). L'utilisation d'une boucle peut sembler naturelle mais n'est pas optimale en temps de calcul. Il est préférable d'utiliser des tests comme dans les exemples ci-dessous.

NB : D'une manière générale, il faut éviter au maximum d'utiliser les boucles quand on travaille avec **R**.

```r

# Extraction d'elements version 1 : fonction which
x_vector <- -5:5  # creation du vecteur
x_vector < 0      # rechercher les elements < 0
index <- which(x_vector < 0) # affecter le resultat au vecteur index
index
x_vector[index]    # extraction des element repondant au critere
x_vector[-index]   # - permet d'acceder aux indices complementaires
# Extraction d'elements version 2 : sans which plus compact mais moins evident
x_vector <- -5:5  # creation du vecteur
index <- x_vector > 0 # le vecteur index est booleen
x_vector[index]    # renvoie le valeurs pour lesquelles index = TRUE
x_vector[x_vector >= 0]   # commande la plus compacte
x_vector[x_vector < 0] <- 10 # remplace les valeurs negatives par 10
```

Listing 7 – Opérations de sélection dans R

Valeurs manquantes. De nombreux jeux de données (séries statistiques) comportent des valeurs manquantes (par exemple en actuariat lorsqu'il manque une certaine information sur un individu. Ces valeurs manquantes sont codées avec la chaîne de caractère 'NA' dans **R**. Cela permet de les identifier et d'utiliser des traitements statistiques adaptés.

```r

# Nettoyage des valeurs manquantes :
x_vector <- 1:10  # creation du vecteur
# NA - Not a Number permet de coder les valeurs manquantes (jeux de donnees incomplets)
x_vector[10] <- NA
x_vector
sum(x_vector)      # renvoie NA car une valeur est manquante
sum(x_vector, na.rm = TRUE) # somme sans les valeurs manquantes
is.na(x_vector)    # verifie si le vecteur contient des valeurs manquantes
which(is.na(x_vector)) # renvoie l'indice de la valeur manquante
# Quelques autres fonctions utiles
A_vector <- rep(1:10, 2)    # cree un vecteur de 1 a 10 2 fois
rep(c(1.4, 3, 5), each = 2) # repete chaque element 2 fois
seq(0, 1, length = 11)  # cree une serie de nombre entre 0 et 1 a pas 2 - 0 / 11 - 1
seq(1.575, 5.125, by = 0.05) # serie de nombres entre deux bornes avec un pas fixe
unique(A_vector)             # supprime les duplications
B_vector <- c(3, 4, 2, 1); sort(B_vector)  # ordonne dans l'ordre croissant
# permutation a effectuer sur les indices pour trier le vecteur
order(B_vector); order(B_vector)[1];  order(B_vector)[length(B_vector)]
```

Lcs - ISI Mahdia / UM

```
20  max(B_vector) ; which.max(B_vector); letters[1:10]
```

Listing 8 – Nettoyage d'une série statistique dans R

VI | Partie pratique

Exercice .1 Soit le vecteur des notes :

$$\textbf{note_vector} = (7, 9, 13, 8, 4, 2, 16, 1, 6, 19, 15, 12, 19, 14, 8, 2, 19, 11, 18, 7)$$

1. Créer le vecteur **note_vector**
2. Donner des commandes qui permettent d'extraire les éléments suivants du vecteur **note_vector**
   a) Le deuxième élément.
   b) Les cinq premiers éléments.
   c) Les éléments strictement supérieurs à 14.
   d) Tous les éléments sauf les éléments en positions 6, 10 et 12.

Exercice .2 À l'aide des fonctions `rep` et `seq` (par exemple on n'écrira pas `c(0,6,0,6,0,6)` pour la première question), créer les vecteurs suivants :

1. 060606
2. 14710
   a) 123123123123
   b) 122333
   c) 111223
   d) 15.510
   e) 111122223333

Exercice .3

1. Créer le vecteur **a_vector** $= (2.5, 3, 1, 0, 4, -1)$
2. Ordonner le vecteur **a_vector** avec la fonction `sort`. Que fait la fonction `order` ? Comment peut-on ordonner **a_vector** en utilisant cette dernière fonction ?

Exercice .4 Étant donné un vecteur d'observations **observation_vector** $= (x_1, x_2, ..., x_N)$ on calcule sa moyenne harmonique selon la formule suivante :

$$M.H. = \frac{N}{\frac{1}{x_1} + \frac{1}{x_2} + \cdots + \frac{1}{x_N}}$$

Le vecteur d'observations est **observation_vector** $= (7, 13, 3, 8, 12, 12, 20, 11)$.

1. Calculer le vecteur $\left(\frac{1}{x_1} + \frac{1}{x_2} + \cdots + \frac{1}{x_N}\right)$.
2. Calculer la moyenne harmonique du vecteur **observation_vector**.

Exercice .5

1. Créer un vecteur **x_vector** $= (x_1, ..., x_N)$ de taille N (on pourra commencer avec $N = 50$) qui contient des nombres au hasard entre 0 et 1 à l'aide de la fonction `runif`.
2. Créer un vecteur **y_vector** de même taille que **x_vector** qui est tel que :
   $\textbf{y_vector}_i = 0$ si $\textbf{x_vector}_i < 0.5$,
   $\textbf{y_vector}_i = 1$ si $\textbf{x_vector}_i \geq 0.5$.
3. En déduire le nombre puis la proportion des coeffcients du vecteur **x_vector** qui sont supérieurs à 0.5. Recommencer l'expérience en simulant plusieurs fois l'échantillon **x_vector** et en testant différentes valeurs de N. Les résultats obtenus sont-ils conformes à votre intuition ?

Exercice .6

1. Créer un vecteur **x_vector** de taille N (on pourra commencer avec $N = 50$) qui est tel que $\textbf{x_vector}_i = \frac{1}{i}$ pour $i \in \{1, \cdots N\}$.
2. En déduire la valeur de

$$\textbf{y_vector}_N = \sum_{i=1}^{N} \frac{1}{i} - \ln(N)$$

pour $N = 50$ puis $N = 100$ et $N = 1000$. La suite $(\textbf{y_vector}_N)$ suite converge t'elle lorsque $N \to +\infty$ (on ne demande pas de prouver le résultat !). Si oui, proposer une valeur approchée de la limite (avec une seule ligne de commande !).

Exercice .7

On admet que

$$\pi = 4 \sum_{k=0}^{+\infty} \frac{(-1)^k}{2k+1}$$

1. En déduire une méthode permettant de fournir une valeur approchée de π. Tester votre méthode en vérifiant que vous arrivez à retrouver les 6 premières décimales de π (la commande **R**, `pi` permet d'obtenir une valeur approchée de π).

Remarque 1 N'hésiter pas à visiter le **site officiel de R Project**, et découvrir la documentation disponible : https ://www.r-project.org/.

(C) Dr. ABBASSI A. Department of Physics - ISI Mahdia / UM